ECE 661: Homework #2
Construct, Train, and Optimize CNN Models

Yiran Chen

ECE Department, Duke University — Fall, 2026

Objectives

Homework #2 covers the contents of Lectures 05∼08. This assignment includes basic knowledge about CNNs, detailed instructions on how to set up a training pipeline for training image classifiers on the CIFAR-10 dataset, how to improve the training pipeline, and how to use advanced CNN architectures to improve the performance of image classifiers. In this assignment, you will gain hands-on experience training a neural network model on a real computer-vision dataset (i.e., CIFAR-10), while also learning techniques for improving the performance of your CNN model.

We encourage you to complete Homework #2 on the JupyterLab server or Google Colab since the model training will require the computing power of GPUs. When conducting the lab projects, actively referring to the **NumPy/PyTorch tutorial** slides on Canvas for instructions on the environment setup and NumPy/PyTorch utilities can be very helpful.

Warning: You are asked to complete the assignment independently.

This lab has 100 points plus 10 bonus points, yet your final score cannot exceed 100 points. You must submit your report in PDF format and your original codes for the lab questions through Gradescope before **11:55pm, Wednesday, September 30**. We provide a template named `simplenn-cifar10.ipynb` to start with, and you are asked to develop your own code based on this template. You will need to submit five independent files including:

1. A self-contained PDF report, which provides answers to all the conceptual questions and clearly demonstrates all your lab results and observations.
2. Three jupyter notebooks for the three labs:
   `simplenn-cifar10.ipynb`,
   `simplenn-cifar10-dev.ipynb`,
   `resnet-cifar10.ipynb`.
3. `predictions.csv`, your predicted label for each image in the provided CIFAR-10 testing split. See detailed instructions in Lab (3).

Keep the cell outputs in your submitted notebooks. **Every number and curve in your report must appear in a notebook output**, otherwise that item receives no credit. **Note that 20 percent of the grade will be deducted if the submission doesn't follow the above guidance. Remember, do NOT generate PDF from your jupyter notebook to serve as the report, which can increase the TA's burden of grading.**

Note that TAs reserve the right to adjust grading based on the submitted homework. We make sure that the grading rule is consistent among all students. Also, the results given for the Labs (for example the reported accuracies) are obtained from the specific runtime when TAs were working on the answers. We do not expect you to get exactly the same numbers; yet, it is necessary that your results show the same trends/patterns/observations in order to receive full credits.

1 True/False Questions (15 pts)

For each question, please provide a short explanation to support your judgment.

Problem 1.1 (3 pts) Any type of data augmentation technique is always beneficial for deep learning applications.

Problem 1.2 (3 pts) Like batch normalization, dropout is a technique for speeding up the convergence of CNN training.

Problem 1.3 (3 pts) Dropout is a common technique to combat overfitting. If L-norm (L1/L2) regularization is used alongside dropout, the performance will always be even better.

Problem 1.4 (3 pts) During training, the Lasso (L1) regularizer causes the model to have a higher sparsity compared to the Ridge (L2) regularizer.

Problem 1.5 (3 pts) The shortcut connections in ResNets result in a smoother loss surface.

2 Computation Questions (15 pts)

For each question, provide the result and show the reasoning.

Problem 2.1 (3 pts) Consider a 100x100 RGB image as an input. If the first hidden layer consists of 100 neurons, and each neuron is fully connected to the input, how many parameters does this hidden layer have including both the weights and the bias parameters?

Problem 2.2 (3 pts) Consider the same 100x100 RGB image as an input. If you use a convolution layer with 100 filters each with size 3x3, how many parameters does this hidden layer have including both the weights and the bias parameters ?

Problem 2.3 (3 pts) Consider an input volume with dimensions 100x100x16, how many parameters does a single 1x1 convolution filter have including the bias term ?

Problem 2.4 (3 pts) Consider the input volume of 100x100x16, and we apply convolution with 32 filters each 5x5 size with a stride of 1 and no padding. What is the output shape?

Problem 2.5 (3 pts) MobileNets use depthwise separable convolution to improve the model efficiency. If we replace all the 3x3 convolution layers in a ResNet architecture with 3x3 depthwise separable convolution layers, by approximately what factor is the number of multiply-add operations of these layers reduced? Assume the number of channels is large.

3 Lab (1): Training SimpleNN for CIFAR-10 classification (15+4 pts)

Just like in HW1, here we start with a simple CNN architecture which we term SimpleNN. It is composed of 2 CONV layers, 2 POOL layers and 3 FC layers. The detailed structure of this model is shown in Table 1.

In this lab, beyond model implementation, you will learn to set up the whole training pipeline and actually train a classifier to perform image classification on the CIFAR-10 dataset [1]. CIFAR-10 is one of the most famous/popular benchmarks for image recognition/classification. It consists of 10 categories (e.g.,

Name	Type	Kernel size	depth/units	Activation	Strides
Conv 1	Convolution	5	8	ReLU	1
MaxPool	MaxPool	2	N/A	N/A	2
Conv 2	Convolution	3	16	ReLU	1
MaxPool	MaxPool	2	N/A	N/A	2
FC1	Fully-connected	N/A	120	ReLU	N/A
FC2	Fully-connected	N/A	84	ReLU	N/A
FC3	Fully-connected	N/A	10	None	N/A

Table 1: SimpleNN structure. No padding is applied on both convolution layers. A flatten layer is required before FC1 to reshape the feature.

bird, dog, car, airplane) with 32x32 RGB images. You may go to the official website for more information https://www.cs.toronto.edu/~kriz/cifar.html.

In this assignment, please refer to Jupyter Notebook `simplenn-cifar10.ipynb` for detailed instructions on how to construct a training pipeline for the SimpleNN model. **Note: remember to unzip the provided `tools.zip` to your workspace before getting started.**

(a) (2 pts) As a sanity check, we should verify the implementation of the SimpleNN model at **Step 0**. How can you check whether the model is implemented correctly?

Hint: 1) Consider creating dummy inputs that are of the same size as CIFAR-10 images, passing them through the model, and see if the model's outputs are of the correct shape. 2) Count the total number of parameters of all conv/FC layers and see if it meets your expectation.

(b) (2 pts) Data preprocessing is crucial to enable successful training and inference of DNN models. Specify the preprocessing functions at **Step 1** and briefly discuss what operations you use and why.

(c) (2 pts) During the training, we need to feed data to the model, which requires an efficient data loading process. This is typically achieved by setting up a `dataset` and a `dataloader`. Please go to **Step 2** and build the actual training/validation datasets and dataloaders. Note, instead of using the `CIFAR10` dataset class from `torchvision.datasets`, here you are asked to use our own CIFAR-10 dataset class, which is imported from `tools.dataset`. As for the dataloader, we encourage you to use `torch.utils.data.DataLoader`.

(d) (2 pts) Go to **Step 3** to instantiate and deploy the SimpleNN model on GPUs for efficient training. How can you verify that your model is indeed deployed on GPU? (Hint: run `!nvidia-smi` in a notebook cell, or print `next(model.parameters()).device`)

(e) (2 pts) Loss functions are used to encode the learning objective. Now, we need to define this problem's loss function as well as the optimizer which will update our model's parameters to minimize the loss. In **Step 4**, please fill out the loss function and optimizer part.

(f) (2 pts) Please go to **Step 5** to set up the training process of SimpleNN on the CIFAR-10 dataset. Follow the detailed instructions in **Step 5** for guidance.

(g) (3 pts) You can start training now **with the provided hyperparameter setting**. What is the initial loss value before you conduct any training step? How is it related to the number of classes in CIFAR-10? What can you observe from **training accuracy** and **validation accuracy**? Do you notice any problems with the current training pipeline?

(h) (**Bonus**, 4 pts) Currently, we do not decay the learning rate during the training. Try to decay the learning rate (you may play with the `DECAY_EPOCHS` and `DECAY` hyperparameters in Step 5). What can you observe compared with no learning rate decay?

At the end of Lab 1, we expect at least 65% validation accuracy if all the steps are completed properly. You are required to submit the completed version of `simplenn-cifar10.ipynb` for Lab (1).

4 Lab (2): Improving the training pipeline (35+6 pts)

In Lab (1), we developed a simplified training pipeline. To obtain better training result, we will improve the training pipeline by employing data augmentation, improving the model design, and tuning the hyperparameters.

Before starting, please duplicate the notebook in Lab (1) and name it as `simplenn-cifar10-dev.ipynb`**, and work on the new notebook**. Your goal is to reach at least 70% validation accuracy on the CIFAR-10 dataset.

(a) (6 pts) Data augmentation techniques help combat overfitting. A typical strategy for CIFAR classification is to combine 1) *random cropping* with a *padding* of 4 and 2) *random horizontal flipping*. Train a model with such augmentation. How does the validation accuracy compare with the one without augmentation? **Note that all the following questions use augmentation. Also remember to reinitialize the model whenever you start a new training!**

(b) (15 pts) Model design is another important factor in determining performance on a given task. Now, modify the design of SimpleNN as instructed below:

- (5 pts) Add a batch normalization (BN) layer after each convolution layer. Compared with no BN layers, how does the best validation accuracy change?
- (5 pts) Use empirical results to show that batch normalization allows a larger learning rate.
- (5 pts) Implement Swish [2] activation on your own, and replace all of the ReLU activations in SimpleNN with Swish. Train the model with BN layers and a learning rate of 0.1. Does Swish outperform ReLU?

(c) (14 pts) Hyperparameter settings are very important and can have a large impact on the final model performance. Based on the improvements that you have made to the training pipeline thus far (with data augmentation, BN layers, and ReLU activations), tune some of the hyperparameters as instructed below:

- (7 pts) Apply different learning rate values: 1.0, 0.1, 0.05, 0.01, 0.005, 0.001, to see how the learning rate affects the model performance, and report results for each. Is a large learning rate beneficial for model training? If not, what can you conclude from the choice of learning rate?
- (7 pts) Use different L2 regularization strengths of 1e-2, 1e-3, 1e-4, 1e-5, and 0.0 to see how the L2 regularization strength affects the model performance. In this problem use a learning rate of 0.01. Report the results for each regularization strength value along with comments on the importance of this hyperparameter.
- (Bonus, 6 pts) Switch the regularization penalty from L2 penalty to L1 penalty. This means you may not use the `weight_decay` parameter in PyTorch builtin optimizers, as it does not support L1 regularization. Instead, you need to add L1 penalty as a part of the loss function. Compare the distribution of weight parameters after L1/L2 regularization. Describe your observations.

Up to now, you should have an improved training pipeline for CIFAR-10. Remember, you are required to submit `simplenn-cifar10-dev.ipynb` for Lab (2).

5 Lab (3): Advanced CNN architectures (20 pts)

The improved training pipeline for SimpleNN developed in Lab (2) still has limited performance. This is mainly because the SimpleNN has rather small capacity (learning capability) for the CIFAR-10 task. Thus, in this lab we replace the SimpleNN model with a more advanced ResNet [3] architecture. We expect to see much higher accuracy on CIFAR-10 when using ResNets. **Here, you may duplicate your jupyter notebook for Lab (2) as** `resnet-cifar10.ipynb` **to serve as a starting point.**

(a) (8 pts) Implement the ResNet-20 architecture by following Section 4.2 of the ResNet paper [3]. This lab is designed to have you learn how to implement a DNN model yourself, **so do NOT borrow any code from online resources.**

(b) (12 pts) Tune your ResNet-20 model to reach an accuracy higher than 90% on the validation dataset. You may use all of the previous techniques that you have learned so far, including data augmentations, hyperparameter tuning, learning rate decay, etc. Training the model longer is also essential to obtaining good performance. You should be able to achieve >90% validation accuracy with a maximum of 200 epochs. **Remember to save your trained model during training!** Check out this tutorial https://pytorch.org/tutorials/beginner/saving_loading_models.html on model saving/loading.

We will grade this task by evaluating your trained model on the holdout testing dataset (for which you do not have labels). **After your ResNet-20 model is trained, you need to make predictions on test data, and save the predictions into the** `predictions.csv` **file**. Please use `save_test_predictions.ipynb` to save your predictions in the required format. The saved file should look like the provided example `sample_predictions.csv`. Upon submission, we will directly compare your predicted labels with the ground-truth labels to compute your score.

After completing Lab (3), you are required to submit `resnet-cifar10.ipynb` and your prediction results `predictions.csv`.

Info: Additional requirements:

- **DO NOT** train on the test set or use pretrained models to get an unfair advantage. We have conducted a special preprocessing on the original CIFAR-10 dataset. As we have tested, training on the public CIFAR-10 labels or using a pretrained CIFAR-10 model does not transfer to our test set and results in a failing score for this lab.
- **DO NOT** copy code from online sources or from classmates. We will check! The consequences can be severe if your code fails our check.

Info: As this assignment requires much computing power of GPUs, we suggest:

- Plan your work in advance and start early. We will **NOT** extend the deadline because of the unavailability of computing resources.
- Be considerate and kill Jupyter Notebook instances when you do not need them.
- **DO NOT** run your program forever. Please follow the recommended/maximum training budget in each lab.

References

[1] A. Krizhevsky, G. Hinton, *et al.*, "Learning multiple layers of features from tiny images," 2009.

[2] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.